

Chapter 71

SQL Question Bank

Answer the SQL questions that come up across screening calls, live-coding rounds, and take-home exercises for every data role · reason through the classic NULL and join traps that catch experienced candidates, not just beginners · write window-function solutions to the "second highest," "top N...

Approved chapter · v1

Contents

Chapter 71. SQL Question Bank

- 71.1 Core concepts: SELECT, WHERE, and JOIN
- 71.2 Basic questions that are trickier than they look
- 71.3 Aggregation: GROUP BY and HAVING
- 71.4 DDL, constraints, and views
- 71.5 Subqueries, CTEs, and EXISTS vs. IN
- 71.6 Window functions: the classics
- 71.7 Classic problems: duplicates, gaps-and-islands, retention
- 71.8 Transactions, ACID, and concurrency
- 71.9 PostgreSQL vs. MySQL: where the dialects actually differ
- 71.10 Query optimization and indexes
- 71.11 Live-coding walk-throughs
- Common mistakes and how to spot them
- In the real world: the interview that hinged on one word
- Tools
- The project
- Final-week revision list
- Key terms
- Where this leads

Chapter 71. SQL Question Bank

Part VIII — The Interview Playbook

You will learn to: answer the SQL questions that come up across screening calls, live-coding rounds, and take-home exercises for every data role · reason through the classic NULL and join traps that catch experienced candidates, not just beginners · write window-function solutions to the "second highest," "top N per group," "running total," "streak," and "retention" problems that recur across companies · know exactly where PostgreSQL and MySQL disagree · walk out with a 20-question final-week revision list.

How this chapter is built. Same format as Chapters 70 and 72A: every core question leads with a "Remember it as..." memory hook, a one-line answer, and a compact tier table. Every rapid-fire section is a scan table. **Every single query in this chapter was actually run against real PostgreSQL 16 and MySQL 8.0 databases**, loaded with Riverstone's practice data (the same `riverstone / riverstone_2025` databases Chapters 12–13 use), not written from memory and not simulated. Where a result looks surprising (an empty result set, a tied ranking, an index the planner declines to use), that's the real database's real answer, kept exactly as returned.

Learn it in pointers reference Chapters 12 and 13 at chapter level (this chat doesn't have their approved text to check exact section numbers against, though their section list was checked directly against the actual manuscript to confirm the topics exist where claimed).

71.1 Core concepts: SELECT, WHERE, and JOIN

Q71-001 · Find customers who have never placed an order

Remember it as: LEFT JOIN + IS NULL, or NOT EXISTS. Never NOT IN if the other side might have a NULL.

Answer in one line:

```
SELECT c.customer_id, c.customer_name
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

Verified on Riverstone: **1 of 24 customers** has never ordered.

Tier	What to say
Passes	WHERE customer_id NOT IN (SELECT customer_id FROM orders) : works on clean data, has a hidden flaw (Q71-002)
Strong	The LEFT JOIN ... IS NULL version above, filtering on the <i>right table's primary key</i> , not an arbitrary column

Tier	What to say
Extra points	+ [Clarify] should a customer whose only order was cancelled count as "never ordered"? + [Validate] total customers = customers-with-orders + customers-without, checked: 24 = 23 + 1 ✓ + [Business] add <code>signup_date</code> and sort by recency, so sales can follow up the newest non-converters first

Likely follow-ups: No orders in the last 90 days, not ever? Products never sold? Rewrite without a join? **Red flag:** `WHERE o.customer_id = NULL` (should be `IS NULL`); filtering a left join's right-table column in `WHERE` in a way that silently turns it into an inner join. **Learn it in:** Chapter 12.

Q71-002 · Why is `NOT IN` dangerous with a subquery that might contain a NULL?

Remember it as: One NULL in a `NOT IN` subquery poisons the whole comparison: the query returns nothing, silently.

Answer in one line: If the subquery's result set contains even one NULL, `NOT IN` returns zero rows for every outer row, because SQL can't prove any value is "not equal to NULL": the comparison evaluates to UNKNOWN, not TRUE, and `NOT IN` needs every comparison to be TRUE.

Verified, live: Riverstone's `orders.sales_rep_id` has 11 NULL rows (unassigned orders).

```
-- returns 0 rows: the trap, live
SELECT count(*) FROM employees WHERE employee_id NOT IN (SELECT sales_rep_id FROM orders);
-- returns 2: the same question, asked correctly
SELECT count(*) FROM employees WHERE employee_id NOT IN
(SELECT sales_rep_id FROM orders WHERE sales_rep_id IS NOT NULL);
```

Tier	What to say
Passes	" <code>NOT IN</code> can behave oddly with NULLs" (vague, no mechanism)
Strong	The UNKNOWN-comparison explanation above, with the fix (<code>WHERE ... IS NOT NULL</code> inside the subquery)
Extra points	+ [Validate] the exact live numbers above: 0 rows (wrong) vs. 2 rows (right) on the same data, same question + [Trade-offs] <code>NOT EXISTS</code> and <code>LEFT JOIN ... IS NULL</code> don't have this problem at all, since they compare row existence, not a value against a NULL-containing set: prefer them by default

Likely follow-ups: Does `IN` (not `NOT IN`) have the same problem? (No: `IN` with a NULL in the list just doesn't match the NULL, it doesn't zero out the whole result.) How would you find this bug in someone else's query? **Red flag:** not knowing this is a real, live trap, not a theoretical one: 11 real NULL rows just demonstrated it. **Learn it in:** Chapter 12.

Rapid-fire, 71.1

#	Question	One-line answer	Extra point
Q71-003	WHERE vs. HAVING ?	Filters rows before grouping / filters groups after aggregation	[Edge cases] can't reference an aggregate function in <code>WHERE</code> : that's precisely what <code>HAVING</code> is for
Q71-004	INNER JOIN vs. LEFT JOIN?	Only matching rows from both tables / all left-table rows, NULLs where there's no match	[Business] a LEFT JOIN that unexpectedly shrinks row count usually means a filter on the right table leaked into <code>WHERE</code> instead of <code>ON</code>

#	Question	One-line answer	Extra point
Q71-005	What does <code>SELECT DISTINCT</code> actually do?	Removes duplicate <i>entire rows</i> from the result, not duplicates in one column alone	[Edge cases] <code>SELECT DISTINCT a, b</code> dedupes on the combination of a and b, not each independently
Q71-006	<code>UNION</code> vs. <code>UNION ALL</code> ?	Removes duplicate rows across the combined result (a real, verified example: 2 identical rows → 2 with <code>UNION ALL</code> , collapses to fewer with <code>UNION</code>) / keeps everything, no dedup check	[Scale] <code>UNION ALL</code> is faster, since it skips the dedup pass: use it whenever you know the two sets are already disjoint
Q71-007	<code>BETWEEN</code> : is it inclusive or exclusive?	Inclusive on both ends	[Edge cases] <code>BETWEEN '2025-01-01' AND '2025-01-31'</code> on a <code>TIMESTAMP</code> column can silently miss same-day rows with a time component after midnight: cast to <code>DATE</code> first, or use explicit <code><</code> on the next day
Q71-008	What's a correlated subquery?	A subquery that references a column from the outer query, re-evaluated once per outer row (e.g., each customer's own most recent order date)	[Trade-offs] correlated subqueries can be slow at scale (one execution per row); a window function or a join often expresses the same logic faster

71.2 Basic questions that are trickier than they look

Every one of these sounds like a warm-up question. Every one of them has caught experienced candidates, because the trap is in something they assumed rather than something they didn't know. Mixed theory and practical, live-verified where a query could actually demonstrate the point.

Q71-047 • Does `NULL = NULL` evaluate to true?

Remember it as: `NULL` isn't a value, it's the absence of one. You can't ask whether two absences are "equal."

Answer in one line: No: `NULL = NULL` evaluates to `NULL` (unknown), not `TRUE` , which is exactly why `WHERE column = NULL` never matches anything and `IS NULL` exists as a separate operator.

Verified, live:

```
SELECT (NULL = NULL) AS result, (NULL = NULL) IS NULL AS is_this_null;
```

```
result | is_this_null
-----+-----
      | t
```

Tier	What to say
Passes	"NULL equals NULL" (the actual trap, stated as fact)
Strong	Correctly says no, it evaluates to NULL/unknown, and explains why <code>= NULL</code> in a <code>WHERE</code> clause silently matches nothing

Tier	What to say
Extra points	+ [Validate] the query above proves it two ways at once: the comparison itself shows blank (NULL), and wrapping it in <code>IS NULL</code> confirms that blank really is NULL, not empty text + [Business] this is exactly the mechanism behind Q71-002's <code>NOT IN</code> trap: one root cause, two different symptoms

Likely follow-ups: How do you check for NULL correctly? What does `NULL AND FALSE` evaluate to? (FALSE: one known-false input is enough to decide an AND, even with an unknown on the other side.)

Red flag: confidently stating `NULL = NULL` is true. **Learn it in:** Chapter 12.

Q71-048 · Can you use a column alias from `SELECT` inside the same query's `WHERE` clause?

Remember it as: `WHERE` runs before `SELECT` ever names anything. You can't filter on a name that doesn't exist yet.

Answer in one line: No: SQL's logical execution order runs `FROM` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `ORDER BY`, so a `WHERE` clause runs before `SELECT` has assigned any alias, and referencing one errors.

Verified, live:

```
SELECT customer_name, city AS loc FROM customers WHERE loc = 'Mumbai';
```

```
ERROR: column "loc" does not exist
```

Tier	What to say
Passes	"You can't use an alias in <code>WHERE</code> " (correct, no reason given)
Strong	+ explains the logical execution order, and that this is <i>why</i> the alias doesn't exist yet at the point <code>WHERE</code> runs
Extra points	+ [Validate] the real error message above, not a guess at what it might say + [Edge cases] <code>ORDER BY</code> runs <i>after</i> <code>SELECT</code> , so aliases work fine there: the same alias that errors in <code>WHERE</code> is perfectly legal in <code>ORDER BY</code> , which is worth stating explicitly since it looks inconsistent until you know the execution order

Likely follow-ups: Does `GROUP BY` allow an alias? (Yes, in PostgreSQL and MySQL, though not universally across every SQL engine.) How would you filter on a computed value if not in `WHERE`? (A CTE or subquery, filtering the outer layer: the same pattern Q71-019 and Q71-021 use for window functions.) **Red flag:** not knowing SQL has a logical execution order distinct from the order it's written in. **Learn it in:** Chapter 12.

Q71-049 · Why does sorting `'9'`, `'10'`, `'2'` as text give a different order than sorting them as numbers?

Remember it as: Text sorts character by character. "10" starts with "1," which comes before "2" and "9": the number itself is never considered.

Answer in one line: As text, comparison is lexicographic (character by character), so '10' sorts before '2' because '1' < '2' as the very first character; as numbers, ordinary numeric comparison applies and $2 < 9 < 10$ as expected.

Verified, live, same three values, two ways:

```
SELECT val FROM (VALUES ('9'),('10'),('2')) AS t(val) ORDER BY val;  -- text
```

```
val
----
10
2
9
```

```
SELECT val FROM (VALUES (9),(10),(2)) AS t(val) ORDER BY val;      -- integer
```

```
val
----
2
9
10
```

Tier	What to say
Passes	"Text and numbers sort differently" (true, no mechanism)
Strong	The character-by-character explanation above, with the real verified output showing both orders on the identical three values
Extra points	+ [Business] this exact bug shows up whenever a numeric ID or code gets stored as VARCHAR instead of an integer type: order IDs, product SKUs, zip codes: and a report silently sorts "wrong" with no error anywhere + [Validate] cast to confirm: ORDER BY val::int on the text version recovers the numeric order, which is itself proof of what was happening

Likely follow-ups: How would you fix a column that's the wrong type after the fact? What about sorting mixed alphanumeric codes like "A10" vs "A9"? **Red flag:** unable to explain *why* it happens, only that it does. **Learn it in:** Chapter 12.

Q71-050 · What's the difference between DELETE , TRUNCATE , and DROP ?

Remember it as: DELETE removes rows, one at a time, with a WHERE if you want. TRUNCATE empties the whole table, fast, no WHERE. DROP removes the table itself, structure and all.

Answer in one line: DELETE FROM t WHERE ... removes matching rows (or all rows with no WHERE), logged row by row, and can be rolled back inside a transaction; TRUNCATE TABLE t empties the entire table at once, much faster, minimally logged, and resets any auto-increment counter; DROP TABLE t removes the table's structure entirely, data and schema both gone.

Tier	What to say
Passes	"They all delete data"
Strong	The three distinct scopes and behaviors above

Tier	What to say
Extra points	+ [Edge cases] <code>TRUNCATE</code> can't take a <code>WHERE</code> clause: it's all rows or nothing, by design + [Business] the practical stakes are real: running <code>TRUNCATE</code> instead of a <code>DELETE ... WHERE</code> on a production table because a <code>WHERE</code> clause got accidentally left off a copy-pasted command is one of the more common, more painful real incidents in data work, and it's a mistake <code>TRUNCATE</code> 's speed makes <i>harder</i> to catch before it's done, not easier + [Trade-offs] <code>DELETE</code> fires row-level triggers (if any exist) and can be rolled back mid-transaction; <code>TRUNCATE</code> typically does neither

Likely follow-ups: Can `TRUNCATE` be rolled back? (In PostgreSQL, yes, inside a transaction; in MySQL, no: it implicitly commits.) What happens to foreign-key-referenced rows on each? **Red flag:** treating all three as interchangeable ways to "clear a table." **Learn it in:** Chapter 12.

Q71-051 • Does a `UNIQUE` constraint allow more than one `NULL` value in a column?

Remember it as: `NULL` never equals `NULL` (Q71-047): so a `UNIQUE` constraint, which is really an equality check, can't call two `NULL`s "duplicates" of each other.

Answer in one line: Yes, in both PostgreSQL and MySQL: a `UNIQUE` constraint allows any number of `NULL` values in that column, because uniqueness is checked by equality, and no two `NULL`s are ever considered equal to each other.

Verified, live:

```
CREATE TEMP TABLE test_unique (id INT, email VARCHAR(50) UNIQUE);
INSERT INTO test_unique VALUES (1, NULL);
INSERT INTO test_unique VALUES (2, NULL); -- succeeds, not rejected
SELECT * FROM test_unique;
```

```
id | email
----+-----
 1 | 
 2 |
```

Both inserts succeeded: a genuinely surprising result to anyone who reads "unique" as "no duplicates, including blanks."

Tier	What to say
Passes	"UNIQUE means no duplicates" (true for non- <code>NULL</code> values, misses the <code>NULL</code> exception entirely)
Strong	Correctly states multiple <code>NULL</code> s are allowed, ties it back to <code>NULL = NULL</code> being unknown, not true
Extra points	+ [Validate] the live proof above: two <code>NULL</code> emails, one <code>UNIQUE</code> constraint, zero errors + [Business] this is precisely why a "unique email" constraint alone doesn't stop two customer records both missing an email address from silently coexisting: a data-quality gap that looks like it should be prevented by the schema and isn't

Likely follow-ups: How would you actually prevent multiple blank emails if that's the real requirement? (A *partial/filtered unique index*, or a *`NOT NULL` constraint alongside `UNIQUE`*, depending on the engine.) Does a `PRIMARY KEY` allow `NULL`s the same way? (No: a primary key is `UNIQUE` plus *`NOT NULL`*, so it never allows even one `NULL`.) **Red flag:** confidently claiming a `UNIQUE` column can never have more than one blank value. **Learn it in:** Chapter 12.

Q71-052 · What happens if you self-join a table without giving it two different aliases?

Remember it as: *The database can't tell which "employees" you mean if you never gave it two different names to tell them apart.*

Answer in one line: It errors: referencing the same table twice with no aliases makes every column reference ambiguous, since the database has no way to know which occurrence of the table you mean.

Verified, live:

```
SELECT employee_id FROM employees JOIN employees ON employee_id = manager_id;
```

```
ERROR:  table name "employees" specified more than once
```

Tier	What to say
Passes	"You need aliases for a self-join" (correct, no example of what breaks without one)
Strong	The explanation above, plus the correctly-aliased version (<code>FROM employees e JOIN employees m ON e.manager_id = m.employee_id</code> , exactly Chapter 13's own self-join, reused in this chapter's Q71-015-adjacent hierarchy work)
Extra points	+ [Validate] the real error message above, which is a different, earlier failure than the "ambiguous column" error a beginner might expect: worth knowing the actual message so you recognize it instantly rather than debugging blind

Likely follow-ups: Would this also happen with a regular join between two *different* tables that happen to share a column name? (*Only for the shared column references, not the table reference itself: a different, related but distinct error.*) **Red flag:** not immediately recognizing a self-join as needing two aliases, or confusing this error with the more familiar "ambiguous column" one. **Learn it in:** Chapter 12 and 13.

Q71-053 · Why does `SELECT *` with a `GROUP BY` often error, even though it "looks" fine?

Remember it as: `GROUP BY` collapses many rows into one per group. `SELECT *` asks for every column: including ones that could hold many different values within that one group.

Answer in one line: Every non-aggregated column in `SELECT` must also appear in `GROUP BY` (in PostgreSQL and in MySQL's stricter modes), because the database can't know which single value to show for a column that wasn't grouped and might differ across the rows being collapsed into one.

Verified, live:

```
SELECT * FROM orders GROUP BY customer_id;
```

```
ERROR:  column "orders.order_id" must appear in the GROUP BY clause or be used in an aggregate function
```

Tier	What to say
Passes	" <code>SELECT *</code> doesn't work well with <code>GROUP BY</code> " (true, no reason)
Strong	The collapsing-rows explanation above, plus the real error naming the exact offending column
Extra points	+ [Edge cases] older MySQL, in its default (non-strict) SQL mode, doesn't error here at all: it silently picks an arbitrary row's value for the ungrouped column, which is far more dangerous than an error, since the query "works" and returns a plausible-looking, unreliable number + [Business] this is a genuine, real reason <code>SELECT *</code> is a bad habit specifically in any query involving <code>GROUP BY</code> ; list only the columns you actually need and can justify being there

Likely follow-ups: What SQL mode controls this behavior in MySQL? How would you fix this query to actually work? **Red flag:** not knowing MySQL's default behavior here can silently differ from PostgreSQL's strict error. **Learn it in:** Chapter 12.

Rapid-fire, 71.9: more basic-but-tricky theory and practice

#	Question	One-line answer	Extra point
Q71-054	Is SQL case-sensitive?	Keywords (<code>SELECT</code> , <code>WHERE</code>) are not case-sensitive; whether your <i>data</i> comparisons are case-sensitive depends on the column's collation, which varies by engine and setup	[Edge cases] <code>WHERE name = 'sharma'</code> may or may not match <code>'Sharma'</code> depending entirely on collation: never assume either way without checking
Q71-055	CHAR vs. VARCHAR ?	<code>CHAR(n)</code> is fixed-length, padded with spaces to exactly n characters; <code>VARCHAR(n)</code> is variable-length, up to n characters, no padding	[Business] <code>CHAR</code> padding can silently break an exact-string comparison against a value that wasn't padded the same way from another source
Q71-056	Do you need a semicolon at the end of a SQL statement?	Required to separate multiple statements in one script; often optional for a single statement in an interactive client	[Edge cases] omitting it inside a multi-statement script can merge two statements into one, garbled query, rather than erroring cleanly
Q71-057	Is a FOREIGN KEY required for a JOIN to work?	No: a <code>JOIN</code> is just a condition on <code>ON</code> ; it works with or without a declared foreign-key constraint	[Business] a missing foreign key doesn't stop bad joins from happening, it just stops the database from <i>rejecting</i> orphaned data before the join ever runs: the constraint is a data-quality guard, not a join requirement
Q71-058	Can two rows be "duplicates" if they have different primary keys?	Yes: a primary key only guarantees that column is unique; every other column can be identical across two rows	[Edge cases] this is exactly why "remove duplicates" almost always means "duplicates ignoring the ID column," which needs to be said explicitly, not assumed
Q71-059	Does <code>LIKE '%text%'</code> match case-sensitively?	Depends on the column's collation, same root cause as Q71-054; PostgreSQL offers <code>ILIKE</code> specifically for a guaranteed case-insensitive match	[Practical] when in doubt, wrap both sides in <code>LOWER()</code> for a comparison that's insensitive regardless of collation
Q71-060	What does an empty string (<code>''</code>) compare as against <code>NULL</code> ?	They are not the same thing: <code>'' IS NULL</code> is false, and <code>'' = NULL</code> is (per Q71-047) unknown/ <code>NULL</code> , never true	[Edge cases] a form that saves a blank text field as <code>''</code> instead of <code>NULL</code> will silently fail every <code>IS NULL</code> check written to catch missing data

71.3 Aggregation: GROUP BY and HAVING

Q71-009 · Total quantity sold per category, only categories over 500 units

Remember it as: *WHERE filters rows before the grouping happens. HAVING filters groups after.*

Answer in one line:

```
SELECT p.category, SUM(oi.quantity) AS total_qty
FROM order_items oi JOIN products p ON oi.product_id = p.product_id
GROUP BY p.category
HAVING SUM(oi.quantity) > 500
ORDER BY total_qty DESC;
```

Verified: Storage 5,350 · Kitchen 3,675 · Industrial 625 (Furniture fell below 500 and was correctly excluded).

Tier	What to say
Passes	Filters with <code>WHERE SUM(quantity) > 500</code> : errors, since aggregates can't be referenced in <code>WHERE</code>
Strong	The <code>GROUP BY + HAVING</code> version above
Extra points	+ [Validate] the three categories shown, and only three, out of four total categories: Furniture is correctly missing, not silently dropped by a bug + [Business] this exact shape (group, sum, filter on the sum) is the core pattern behind almost every "which segments matter" business question

Likely follow-ups: Same, but only the top 2 categories regardless of a fixed threshold? Add a customer-segment breakdown too? **Red flag:** trying to filter an aggregate in `WHERE` . **Learn it in:** Chapter 12.

Rapid-fire, 71.2

#	Question	One-line answer	Extra point
Q71-010	Does <code>COUNT(*)</code> differ from <code>COUNT(column)</code> ?	<code>COUNT(*)</code> counts all rows including NULLs in any column / <code>COUNT(column)</code> counts only non-NULL values in that column	[Edge cases] <code>COUNT(sales_rep_id)</code> on Riverstone's orders returns 164, not 175: the 11 NULL rows are silently excluded
Q71-011	Can you <code>GROUP BY</code> a column not in the <code>SELECT</code> list?	Yes, in both PostgreSQL and MySQL: you can group by something you don't display	[Edge cases] the reverse is stricter: every non-aggregated <code>SELECT</code> column generally must appear in <code>GROUP BY</code> , or the query errors (Postgres) or gives an arbitrary value (older MySQL modes)
Q71-012	What does <code>GROUP BY 1, 2</code> mean?	Groups by the 1st and 2nd columns in the <code>SELECT</code> list, by position, not by name	[Trade-offs] convenient for a long expression, but fragile: reordering <code>SELECT</code> columns silently changes what's grouped
Q71-013	How would you get a count of DISTINCT customers per month?	<code>SELECT date_trunc('month', order_date), COUNT(DISTINCT customer_id) FROM orders GROUP BY 1</code>	[Business] distinct customers per month, not order count, is the number that actually answers "is our customer base growing"

71.4 DDL, constraints, and views

Q71-068 · Demonstrate the difference between ON DELETE CASCADE and the default (RESTRICT) behavior

Remember it as: CASCADE says "delete the children too." The default says "refuse to delete the parent while children still exist."

Answer in one line: Without ON DELETE CASCADE , deleting a referenced parent row is rejected outright if any child rows still reference it; with ON DELETE CASCADE , deleting the parent automatically deletes those child rows too.

Verified, live, both behaviors on the same schema shape:

```
-- WITH CASCADE: deleting parent 1 also removes its children automatically
CREATE TEMP TABLE parent_t (id INT PRIMARY KEY);
CREATE TEMP TABLE child_t (id INT, parent_id INT REFERENCES parent_t(id) ON DELETE CASCADE);
INSERT INTO parent_t VALUES (1),(2);
INSERT INTO child_t VALUES (10,1),(11,1),(12,2);
DELETE FROM parent_t WHERE id = 1;
SELECT * FROM child_t;
```

id	parent_id
12	2

(Rows 10 and 11, both children of parent 1, are gone too.)

```
-- WITHOUT CASCADE (the default): the delete itself is blocked
CREATE TEMP TABLE parent_r (id INT PRIMARY KEY);
CREATE TEMP TABLE child_r (id INT, parent_id INT REFERENCES parent_r(id));
INSERT INTO parent_r VALUES (1);
INSERT INTO child_r VALUES (10,1);
DELETE FROM parent_r WHERE id = 1;
```

```
ERROR:  update or delete on table "parent_r" violates foreign key constraint
"child_r_parent_id_fkey" on table "child_r"
DETAIL:  Key (id)=(1) is still referenced from table "child_r".
```

Tier	What to say
Passes	" CASCADE deletes related rows too" (correct, no comparison against the default)
Strong	Both behaviors above, explicitly contrasted: cascade silently removes children; the default blocks the delete with an error
Extra points	+ [Validate] both real outcomes shown on parallel schemas: 2 rows silently gone with CASCADE, 1 delete flatly refused without it + [Business] CASCADE is powerful and dangerous: deleting one customer with CASCADE set on their orders would silently delete their entire order history too, which is rarely what anyone actually wants for a customer record: reserve CASCADE for genuinely dependent data (an order's own line items), not for a parent whose "children" are actually independently valuable records

Likely follow-ups: What's `ON DELETE SET NULL` ? When would you prefer it to `CASCADE`? What about `ON UPDATE CASCADE` ? **Red flag:** treating `CASCADE` as a safe default rather than a deliberate, case-by-case decision. **Learn it in:** Chapter 12.

Q71-069 · Build a view for "each customer's total revenue," and explain what a view actually is under the hood

Remember it as: A view isn't a copy of the data. It's a saved query with a name, re-run fresh every time you query it.

Answer in one line: `CREATE VIEW` saves a query definition under a name; querying the view re-runs that underlying query against the live, current data every time, it doesn't store a snapshot.

Verified, live:

```
CREATE VIEW customer_revenue AS
SELECT c.customer_id, c.customer_name,
       SUM(oi.quantity*oi.unit_price*(1-oi.discount_pct/100)) AS revenue
FROM customers c JOIN orders o ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
WHERE o.status <> 'Cancelled'
GROUP BY c.customer_id, c.customer_name;

SELECT * FROM customer_revenue ORDER BY revenue DESC LIMIT 3;
```

customer_id	customer_name	revenue
1	Sharma Hardware	502775.00
11	Harbour Traders	412680.50
7	Northgate Distributors	353088.50

Tier	What to say
Passes	"A view is like a virtual table" (true, doesn't explain the "re-run every time" mechanic)
Strong	+ explicitly: a plain view has no stored data of its own; every <code>SELECT</code> against it executes the underlying query fresh, so it's always current, and it inherits the underlying query's full cost every single time
Extra points	+ [Validate] the real top-3 customers above, matching what you'd get running the underlying <code>GROUP BY</code> query directly, since that's literally what the view does + [Trade-offs] a materialized view (<code>CREATE MATERIALIZED VIEW</code>) is the opposite trade: it <i>does</i> store the result physically, is fast to query, but goes stale until explicitly refreshed with <code>REFRESH MATERIALIZED VIEW</code> , a real trade between freshness and speed
Business	a view is also a clean way to hide a complex join behind a simple name for less technical users, and to enforce row-level access (a view showing only a rep's own accounts) without duplicating logic across every query that needs it

Likely follow-ups: Can you `INSERT` into a view? (Sometimes, with restrictions, depending on the view's complexity.) What's the performance cost of querying a view built on a view? **Red flag:** believing a plain view stores a copy of the data at creation time. **Learn it in:** Chapter 12.

Rapid-fire, 71.11

#	Question	One-line answer	Extra point
Q71-070	ALTER TABLE ... ADD COLUMN with a default value on a huge existing table, what changed for you to know about?	Modern PostgreSQL (11+) adds a column with a constant default instantly, without rewriting every existing row; older versions, and some other engines, rewrite the whole table, which can lock it for a long time	[Business] always check your specific engine and version before assuming a schema change on a huge production table is "instant"
Q71-071	What's the difference between a PRIMARY KEY and a UNIQUE constraint?	A table can have only one PRIMARY KEY (which also disallows NULL); it can have many UNIQUE constraints, and each one still allows multiple NULLs (Q71-051)	[Edge cases] this is precisely why an email UNIQUE constraint doesn't stop multiple blank emails, but a PRIMARY KEY on the same column would (since it forbids NULL entirely)
Q71-072	What does NOT NULL actually enforce, and can it be added to an existing column with existing data?	Rejects any future NULL in that column; adding it to an existing column requires every current value to already be non-NULL, or the ALTER TABLE itself fails	[Practical] always run a check query first (SELECT COUNT(*) WHERE col IS NULL) before attempting to add NOT NULL to a live table
Q71-073	What's a materialized view refresh strategy, in one sentence?	Either refresh on a schedule (nightly, hourly) or refresh triggered by the underlying data changing, chosen based on how stale the data is allowed to get	[Business] this is the same freshness-vs-cost trade-off as a Power BI Import-mode refresh (Chapter 70, §70.7), just at the database layer instead of the BI layer

71.5 Subqueries, CTEs, and EXISTS vs. IN

Q71-014 · Rewrite a nested subquery as a CTE, and explain why you would

Remember it as: A CTE is a subquery with a name and a place to breathe. Same result, easier to read and debug one step at a time.

Answer in one line: WITH name AS (...) SELECT ... FROM name names an intermediate result so it can be read top-to-bottom and reused, instead of nesting parentheses inside parentheses.

```
WITH monthly_revenue AS (  
  SELECT date_trunc('month', o.order_date)::date AS month,  
         SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct/100)) AS revenue  
  FROM orders o JOIN order_items oi ON o.order_id = oi.order_id  
  WHERE o.status <> 'Cancelled'  
  GROUP BY 1  
)  
SELECT month, revenue, SUM(revenue) OVER (ORDER BY month) AS running_total  
FROM monthly_revenue ORDER BY month;
```

Verified, first three months: Jan ₹202,640 (running ₹202,640) · Feb ₹253,664 (running ₹456,304) · Mar ₹278,007.50 (running ₹734,311.50).

Tier	What to say
Passes	A nested subquery achieving the same result: correct, harder to read past two levels

Tier	What to say
Strong	The CTE version above, one logical step at a time
Extra points	+ [Validate] running total reconciles: 202,640 + 253,664 = 456,304 ✓ + [Scale] multiple CTEs can chain (WITH a AS (...), b AS (...)), each building on the last, which is how a genuinely complex query stays readable

Likely follow-ups: Recursive CTE: when would you need one? Does a CTE get materialized (computed once) or inlined, and does that matter for performance? **Red flag:** nesting subqueries three or four levels deep when a CTE would make the same logic readable. **Learn it in:** Chapter 13.

Q71-015 · Write a recursive CTE to show Riverstone's full management hierarchy

Remember it as: A recursive CTE is a loop: start with the base case (the top of the tree), then repeatedly join back to itself for the next level down.

<pre> WITH RECURSIVE hierarchy AS (SELECT employee_id, employee_name, manager_id, 1 AS level FROM employees WHERE manager_id IS NULL -- base case: the top UNION ALL SELECT e.employee_id, e.employee_name, e.manager_id, h.level + 1 FROM employees e JOIN hierarchy h ON e.manager_id = h.employee_id -- recursive step) SELECT employee_name, level FROM hierarchy ORDER BY level, employee_name; </pre>

Verified: Anita Rao (level 1) → Farah Khan, Vikram Singh (level 2) → Neha Kulkarni, Rahul Mehta (level 3).

Tier	What to say
Passes	Can't produce a working recursive CTE at all; describes the idea in English
Strong	The structure above: a base case with UNION ALL to a self-referencing recursive term
Extra points	+ [Validate] the output above is a real, correctly-ordered 3-level hierarchy on Riverstone's actual employee table + [Edge cases] a recursive CTE with no terminating condition (a cyclic manager relationship) runs forever; PostgreSQL has no automatic depth limit by default: add a level cap for safety on untrusted data

Likely follow-ups: Find everyone reporting up to a specific manager, at any depth? What if the hierarchy has a cycle by mistake? **Red flag:** not knowing the base case / recursive term structure at all. **Learn it in:** Chapter 13.

Rapid-fire, 71.3

#	Question	One-line answer	Extra point
Q71-016	EXISTS vs. IN : when do they differ in behavior?	Both correctly answer "does a match exist" and gave the identical count (23) on a live test here	[Trade-offs] EXISTS stops at the first match per row and doesn't build the full candidate set first, which is often faster on a large subquery; NOT EXISTS is also immune to NOT IN 's NULL trap (Q71-002)

#	Question	One-line answer	Extra point
Q71-017	What's a scalar subquery?	A subquery expected to return exactly one row, one column, used anywhere a single value is expected	[Edge cases] if it returns more than one row, the query errors at runtime, not at write time
Q71-018	Each customer's most recent order date, without a GROUP BY ?	A correlated scalar subquery: <code>SELECT c.customer_name, (SELECT MAX(order_date) FROM orders o WHERE o.customer_id=c.customer_id) FROM customers c</code>	[Trade-offs] correct and simple to read; a <code>LEFT JOIN</code> to a pre-aggregated subquery is usually faster on a large table, since it avoids one execution per outer row

71.6 Window functions: the classics

Q71-019 · Find the second-highest-priced product

Remember it as: `LIMIT/OFFSET` breaks on ties. `DENSE_RANK` doesn't.

Answer in one line:

```
SELECT product_name, unit_price FROM (
  SELECT product_name, unit_price, DENSE_RANK() OVER (ORDER BY unit_price DESC) AS rnk
  FROM products
) t WHERE rnk = 2;
```

Verified: Garden Chair, ₹1,150.00.

Tier	What to say
Passes	<code>ORDER BY unit_price DESC LIMIT 1 OFFSET 1</code> : gives the same right answer here, and silently breaks if two products tie for first place (<code>OFFSET 1</code> would then return one of the <i>tied firsts</i> , not the true second-distinct price)
Strong	The <code>DENSE_RANK</code> version above, explicitly immune to ties
Extra points	+ [Edge cases] <code>RANK</code> vs. <code>DENSE_RANK</code> matters here specifically: <code>RANK</code> would skip a number after a tie (1,1,3), so "rnk = 2" could return zero rows if two products tied for first: <code>DENSE_RANK (1,1,2)</code> doesn't have that gap + [Validate] on Riverstone's real (untied) top prices, both approaches agree, which is exactly why this bug hides until the data changes

Likely follow-ups: Third-highest? Second-highest *per category*, not overall? What if there's a tie for second place itself: should both show? **Red flag:** using `LIMIT / OFFSET` with no acknowledgment that it breaks on ties. **Learn it in:** Chapter 13.

Q71-020 · `ROW_NUMBER`, `RANK`, and `DENSE_RANK` : demonstrate the difference on real tied data

Remember it as: `ROW_NUMBER` never ties (1,2,3). `RANK` ties and skips (1,1,3). `DENSE_RANK` ties and doesn't skip (1,1,2).

Verified, live, on customers tied at 16 orders each (Sharma Hardware and Metro Mart):


```
SELECT c.customer_name, COUNT(*) AS order_count,
       ROW_NUMBER() OVER (ORDER BY COUNT(*) DESC) AS rn,
       RANK() OVER (ORDER BY COUNT(*) DESC) AS rnk,
       DENSE_RANK() OVER (ORDER BY COUNT(*) DESC) AS drnk
FROM orders o JOIN customers c ON o.customer_id = c.customer_id
GROUP BY c.customer_name ORDER BY order_count DESC LIMIT 4;
```

customer_name	order_count	rn	rnk	drnk
Green Leaf Hotels	17	1	1	1
Sharma Hardware	16	2	2	2
Metro Mart	16	3	2	2
Northgate Distributors	13	4	4	3

Tier	What to say
Passes	Names all three functions, can't explain how they differ on a tie
Strong	Reads the table above correctly: RANK gives both tied customers rank 2, then <i>skips</i> to 4 for the next distinct value; DENSE_RANK gives them both 2, then continues at 3 with no gap
Extra points	+ [Validate] this is real tied data, not a constructed example: Sharma Hardware and Metro Mart are genuinely tied at 16 orders each in Riverstone's actual database + [Business] "top 3 customers" means something different depending which function you pick when there's a real tie at the boundary: worth clarifying with whoever asked for the list

Likely follow-ups: Which one would you use for "top N per group" and why? What does NTILE do?

Red flag: claiming they're interchangeable, or unable to predict what happens at a tie without running it. **Learn it in:** Chapter 13.

Q71-021 · Top 2 products by revenue, within each category

Remember it as: PARTITION BY is GROUP BY for window functions: it resets the ranking at each group boundary instead of collapsing rows.

Answer in one line:

```
SELECT category, product_name, revenue FROM (
  SELECT p.category, p.product_name,
         SUM(oi.quantity*oi.unit_price*(1-oi.discount_pct/100)) AS revenue,
         ROW_NUMBER() OVER (PARTITION BY p.category ORDER BY SUM(oi.quantity*oi.unit_price*(1-
oi.discount_pct/100)) DESC) AS rn
  FROM order_items oi JOIN products p ON oi.product_id = p.product_id
  JOIN orders o ON oi.order_id = o.order_id WHERE o.status <> 'Cancelled'
  GROUP BY p.category, p.product_name
) t WHERE rn <= 2 ORDER BY category, rn;
```

Verified:

category	product_name	rn
Furniture	Garden Chair	1
Industrial	Industrial Crate	1
Kitchen	Lunch Box Set	1

category	product_name	rn
Kitchen	Food Container Set	2
Storage	Storage Box 25L	1
Storage	Storage Box 10L	2

Note: **Furniture and Industrial only show one row each**, not two: real data, not a bug. Each of those categories genuinely has only one product with recorded sales in this dataset.

Tier	What to say
Passes	A correlated subquery per category finding the top product, then a second one for the runner-up: works, ugly, doesn't generalize past "top 2"
Strong	The <code>PARTITION BY</code> version above: one query, any N, generalizes cleanly
Extra points	+ [Edge cases] exactly this: two categories return fewer than 2 rows because they only have 1 product with sales: a filter of <code>rn <= 2</code> never guarantees exactly 2 rows per group, and it's worth saying so before the result surprises someone + [Validate] the totals for Kitchen's two shown products, ₹546,630 and ₹523,745, are both individually plausible against the category's ₹3,675 total units moved at typical Kitchen prices

Likely follow-ups: Same but by revenue *share* within category, not rank? What changes using `RANK` instead of `ROW_NUMBER` here? **Red flag:** not anticipating that some groups may have fewer members than N. **Learn it in:** Chapter 13.

Q71-022 · Month-over-month revenue change with `LAG`

Remember it as: `LAG` looks backward one row. `LEAD` looks forward one row. Both need an `ORDER BY` to mean anything.

Answer in one line:

```
WITH monthly_revenue AS (
  SELECT date_trunc('month', o.order_date)::date AS month,
         SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct/100)) AS revenue
  FROM orders o JOIN order_items oi ON o.order_id = oi.order_id
  WHERE o.status <> 'Cancelled'
  GROUP BY 1
)
SELECT month, revenue, revenue - LAG(revenue) OVER (ORDER BY month) AS mom_change
FROM monthly_revenue ORDER BY month;
```

Verified, first four months: Jan (change: `NULL`, nothing before it) · Feb +₹51,024 · Mar +₹24,343.50 · Apr -₹67,726.

Tier	What to say
Passes	A self-join on <code>month = month - 1</code> : works, verbose, breaks if a month is entirely missing from the data (the join simply finds no match, silently)
Strong	<code>LAG</code> as above; explicitly notes the first row's change is <code>NULL</code> , correctly, since there's no prior month

Tier	What to say
Extra points	+ [Validate] the real negative number in April (−₹67,726) is worth pointing out explicitly: a real month-over-month decline, kept as-is rather than picking a friendlier month to showcase + [Edge cases] LAG assumes no gaps in the sequence; a month with zero orders wouldn't appear as a row at all unless the data was first generated to include it, which would silently make the "previous month" for the following row not actually adjacent in time

Likely follow-ups: LAG two periods back instead of one? Percentage change, not absolute? LEAD for "days until next order"? **Red flag:** forgetting ORDER BY inside the window (the function becomes meaningless without a defined row order). **Learn it in:** Chapter 13.

Rapid-fire, 71.4

#	Question	One-line answer	Extra point
Q71-023	What does SUM(x) OVER (ORDER BY month) compute, with no PARTITION BY ?	A running total across all rows, in the specified order	[Edge cases] without an explicit frame clause, the default frame is "start to current row," which is exactly what makes it a running total rather than a grand total
Q71-024	What's the difference between a window function and GROUP BY ?	GROUP BY collapses rows into one per group; a window function keeps every original row and adds a calculated column alongside it	[Business] this is <i>why</i> window functions exist for reporting: you can show a customer's individual order next to their running total, in the same row, which plain aggregation can't do
Q71-025	What's NTILE(4) for?	Splits ordered rows into 4 roughly equal-sized buckets (quartiles)	[Business] the SQL-native way to build a quartile or decile segmentation directly in a query
Q71-026	Can you use a window function's result in the same query's WHERE clause?	No: window functions are evaluated after WHERE ; wrap the query in a subquery or CTE and filter the outer layer instead	[Edge cases] this is exactly why Q71-019 and Q71-021 both wrap the window function in a subquery before filtering

71.7 Classic problems: duplicates, gaps-and-islands, retention

Q71-027 · Find any duplicate customer names

Remember it as: GROUP BY the thing that should be unique, HAVING COUNT > 1.

```
SELECT customer_name, COUNT(*) FROM customers GROUP BY customer_name HAVING COUNT(*) > 1;
```

Verified: 0 rows. Riverstone's real customer table has no exact-name duplicates.

Tier	What to say
Passes	The query above, correctly, with no result to show
Strong	+ explicitly notes that an empty result here is the <i>real</i> answer on this data, not a broken query, and explains how they'd confirm that (re-running with a column known to have duplicates, or checking row counts)

Tier	What to say
Extra points	+ [Edge cases] exact-name matching misses near-duplicates: trailing spaces, case differences, "Sharma Hardware" vs. "Sharma Hardware Pvt Ltd": those need TRIM / LOWER normalization or fuzzy matching first, and are a different, harder problem + [Business] near-duplicate customer records are one of the most common real data-quality issues in a CRM, and this exact-match query is only the first, cheapest check, not the whole answer

Likely follow-ups: How would you catch near-duplicates, not just exact ones? What would you do once you found real duplicates: merge them how? **Red flag:** assuming an empty result means the query is wrong, rather than considering it might be the honest answer. **Learn it in:** Chapter 12.

Q71-028 · Find every customer's longest streak of consecutive order-days

Remember it as: The gaps-and-islands trick: subtract a row number from the date. Rows in the same streak land on the exact same result.

Answer in one line:

```
WITH daily AS (SELECT DISTINCT customer_id, order_date FROM orders),
numbered AS (
  SELECT customer_id, order_date,
         order_date - (ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date))::int AS grp
  FROM daily
)
SELECT customer_id, MIN(order_date) AS streak_start, MAX(order_date) AS streak_end, COUNT(*) AS streak_len
FROM numbered GROUP BY customer_id, grp HAVING COUNT(*) >= 2 ORDER BY streak_len DESC;
```

Verified: two 2-day streaks found (customer 15: Jun 9–10; customer 5: Jan 12–13). No streak in this dataset reaches 3+ consecutive days.

Tier	What to say
Passes	Can describe the goal ("find consecutive dates") but can't produce the row-number-minus-date trick unprompted
Strong	The gaps-and-islands query above, with the mechanism explained: within one true streak, date increases by 1 each row and so does the row number, so their difference stays constant; a real gap breaks that constancy and starts a new group
Extra points	+ [Validate] the real result, honestly reported: only 2-day streaks exist in this dataset, nothing longer, rather than picking a more impressive-looking (fake) example + [Depth] this same "constant-difference" trick generalizes to any gaps-and-islands problem: consecutive login days, consecutive shipped orders, consecutive price increases

Likely follow-ups: Same, but for consecutive *calendar weeks*, not days? What's the equivalent trick if the sequence isn't dates but just row order? **Red flag:** reaching for a procedural loop (a cursor, or code outside SQL) as the only solution they can think of. **Learn it in:** Chapter 13.

Q71-029 · Which customers ordered in both January and February 2025? (a simple retention check)

Remember it as: Two EXISTS clauses, ANDed together: "ordered in month A" AND "ordered in month B."

```
SELECT count(DISTINCT o1.customer_id) FROM orders o1
WHERE EXISTS (SELECT 1 FROM orders o2 WHERE o2.customer_id = o1.customer_id
              AND o2.order_date BETWEEN '2025-01-01' AND '2025-01-31')
AND EXISTS (SELECT 1 FROM orders o3 WHERE o3.customer_id = o1.customer_id
              AND o3.order_date BETWEEN '2025-02-01' AND '2025-02-28');
```

Verified: 4 customers ordered in both January and February 2025.

Tier	What to say
Passes	Two separate queries (Jan customers, Feb customers), compared by eye: works for a one-off check, doesn't scale to "for every pair of consecutive months"
Strong	The double- EXISTS version above, or equivalently two COUNT(CASE WHEN ...) columns compared in one row
Extra points	+ [Depth] this is retention at its simplest: real cohort retention analysis (Chapter 22-adjacent territory) extends this into a full month-by-month grid, typically built with conditional aggregation (SUM(CASE WHEN ... THEN 1 ELSE 0 END)) rather than repeated EXISTS calls + [Business] 4 of Riverstone's ~23 active customers retained month-to-month here is a real, low-feeling number worth a follow-up question in an actual work setting: is that normal for this business, or a problem?

Likely follow-ups: Build a full 12-month retention grid? What's the difference between this and a rolling "active in last 30 days" count? **Red flag:** confusing "ordered in both months" with "ordered at all in the date range" (missing the AND-of-two-EXISTS structure). **Learn it in:** Chapter 13 (and Chapter 22 for retention/cohort analysis proper).

Rapid-fire, 71.5

#	Question	One-line answer	Extra point
Q71-030	Find products that have never been sold?	LEFT JOIN products to order_items, WHERE order_item... IS NULL , or NOT EXISTS	[Validate] verified live: 0 rows: every one of Riverstone's 8 products has at least one recorded sale in this dataset
Q71-031	Find the Nth highest value in general, not just 2nd?	DENSE_RANK() OVER (ORDER BY x DESC) , filter WHERE rnk = N in an outer query	[Edge cases] same tie-safety reasoning as Q71-019, for any N
Q71-032	Compute a percentage of total per row?	x / SUM(x) OVER () * 100 , no GROUP BY needed since the window with no PARTITION BY sees every row	[Business] this is the SQL-native way to build a "% of total" column directly, without a second query or a spreadsheet pivot

71.8 Transactions, ACID, and concurrency

Q71-061 • Demonstrate that ROLLBACK actually undoes a change, not just "cancels" it

Remember it as: Nothing inside BEGIN...COMMIT is real until COMMIT. ROLLBACK makes it as if it never happened at all.

Answer in one line: A transaction's changes are only permanent once COMMIT runs; ROLLBACK reverts every change made since BEGIN , restoring the exact prior state.

Verified, live:

```
CREATE TEMP TABLE tx_test (id INT, val TEXT);
INSERT INTO tx_test VALUES (1, 'original');
BEGIN;
UPDATE tx_test SET val = 'changed' WHERE id = 1;
SELECT * FROM tx_test;  -- shows 'changed', inside the open transaction
ROLLBACK;
SELECT * FROM tx_test;  -- shows 'original' again
```

```
id | val
----+-----
 1 | changed
(1 row)

id | val
----+-----
 1 | original
(1 row)
```

Tier	What to say
Passes	" ROLLBACK undoes changes" (correct, no demonstration of the actual mechanics)
Strong	+ explains that the UPDATE is visible <i>within</i> the same session immediately, but is not durable until COMMIT , and ROLLBACK discards it entirely
Extra points	+ [Validate] the live proof above: the same SELECT , before and after ROLLBACK , gives two different, real answers on the same table + [Business] this is <i>why</i> multi-step operations (move money between two accounts, place an order and decrement stock together) belong inside one transaction: if step two fails, ROLLBACK guarantees step one never silently sticks around half-finished

Likely follow-ups: What does COMMIT do that makes a change durable? What happens if the connection drops before either COMMIT or ROLLBACK runs? (*The transaction is implicitly rolled back.*) **Red flag:** not knowing changes are visible inside the same session before commit, only reverted on ROLLBACK or invisible to *other* sessions until commit. **Learn it in:** Chapter 12.

Q71-062 • What does ACID stand for, and give one real example of each

Remember it as: Atomic: *all or nothing*. Consistent: *rules always hold*. Isolated: *transactions don't see each other's half-finished work*. Durable: *once committed, it survives a crash*.

Answer in one line: Atomicity (a transaction fully succeeds or fully fails, no partial state), Consistency (the database moves from one valid state to another, never violating its own constraints), Isolation (concurrent transactions don't see each other's uncommitted changes), Durability (once committed, a change survives a crash or power loss).

Tier	What to say
Passes	Can name what the letters stand for, can't give a concrete example of any of them
Strong	+ one real example each: Atomicity (Q71-061's transfer example: both the debit and credit happen, or neither does) · Consistency (a CHECK constraint, Q71-063, refusing an invalid state) · Isolation (two people checking stock at once shouldn't both believe the last unit is available) · Durability (a committed order survives the database server crashing five seconds later)

Tier	What to say
Extra points	+ [Depth] Isolation specifically has <i>levels</i> (Q71-064), and "isolated" doesn't mean one single guaranteed behavior; it means a spectrum of trade-offs + [Business] these four properties are why a relational database, not a plain file or a simple key-value store, is the default choice for anything involving money, inventory, or any other value that must never silently go wrong

Likely follow-ups: Which of the four is hardest to guarantee at scale? What database systems intentionally relax some of these, and why? **Red flag:** reciting the acronym with no example for any letter. **Learn it in:** Chapter 12.

Q71-063 · Write a `CHECK` constraint enforcing that a discount percentage is between 0 and 100, and show it actually rejecting bad data

Remember it as: A `CHECK` constraint is a rule the database enforces on every single write, not a rule you hope the application remembers to enforce.

Verified, live:

```
CREATE TEMP TABLE tx_check (id INT, discount_pct NUMERIC CHECK (discount_pct BETWEEN 0 AND 100));
INSERT INTO tx_check VALUES (1, 50);    -- succeeds
INSERT INTO tx_check VALUES (2, 150);   -- rejected
```

```
ERROR:  new row for relation "tx_check" violates check constraint "tx_check_discount_pct_check"
DETAIL:  Failing row contains (2, 150).
```

Tier	What to say
Passes	"You'd validate the discount in the application code before saving it"
Strong	The <code>CHECK</code> constraint above, and why it's a stronger guarantee than application-level validation alone
Extra points	+ [Validate] the real error above, including the <code>DETAIL</code> line showing the exact rejected row + [Business] application-level validation can be bypassed by a second application, a manual data fix, or a bug; a database-level <code>CHECK</code> constraint can't be bypassed by anything that writes to the table, which is exactly Consistency (Q71-062) enforced automatically rather than hoped for

Likely follow-ups: What's the difference between a `CHECK` constraint and a `NOT NULL` constraint? Can a `CHECK` constraint reference another table? (No, *not directly*, a *trigger is needed for cross-table rules*.)

Red flag: relying entirely on application code to enforce a rule the database itself could guarantee.

Learn it in: Chapter 12.

Rapid-fire, 71.10

#	Question	One-line answer	Extra point
Q71-064	What's a transaction isolation level, in plain terms?	A setting controlling how much of another transaction's in-progress, uncommitted work you can "see" while your own transaction is running	[Depth] from loosest to strictest: Read Uncommitted, Read Committed (PostgreSQL and MySQL's usual default), Repeatable Read, Serializable

#	Question	One-line answer	Extra point
Q71-065	What's a "dirty read"?	Reading another transaction's uncommitted change, which might still be rolled back and never actually happen	[Edge cases] only possible at the loosest isolation level (Read Uncommitted); PostgreSQL doesn't actually implement this level distinctly, it behaves as Read Committed even if requested
Q71-066	What's a deadlock?	Two transactions each waiting on a lock the other one holds, so neither can ever proceed	[Business] the database detects this automatically and forcibly rolls back one of the two transactions, rather than letting both hang forever; application code needs to catch that error and retry
Q71-067	Does a <code>SELECT</code> (read-only) ever get blocked by another transaction's write?	Under PostgreSQL and MySQL's usual default isolation level, no, a plain read sees the last <i>committed</i> version and isn't blocked waiting for an in-progress write	[Depth] this is MVCC (multi-version concurrency control): readers and writers don't block each other by default, which is why both engines handle concurrent read-heavy and write-heavy workloads reasonably well out of the box

71.9 PostgreSQL vs. MySQL: where the dialects actually differ

Every pair below was run on both engines against the same Riverstone data.

Q71-033 · String aggregation: `STRING_AGG` VS. `GROUP_CONCAT`

Remember it as: *Same job, different name, different separator syntax.*

PostgreSQL:

```
SELECT category, STRING_AGG(DISTINCT product_name, ' , ' ORDER BY product_name) FROM products GROUP BY category;
```

MySQL:

```
SELECT category, GROUP_CONCAT(DISTINCT product_name ORDER BY product_name SEPARATOR ' , ') FROM products GROUP BY category;
```

Verified identical output on both engines, e.g. Kitchen → "Food Container Set, Lunch Box Set, Water Bottle 1L".

Tier	What to say
Passes	Knows one function exists, not the other engine's name for it
Strong	Both function names and their (slightly different) syntax for the separator
Extra points	+ [Edge cases] MySQL's <code>GROUP_CONCAT</code> has a default length limit (<code>group_concat_max_len</code> , 1024 bytes by default in many configurations) that silently truncates long results: worth knowing before it bites in production

Learn it in: Chapter 12, §12.16 (MySQL dialect).

Rapid-fire, 71.6: dialect differences, all live-verified

#	Feature	PostgreSQL	MySQL	Extra point
Q71-034	NULL-coalescing	<code>COALESCE(x, 'default')</code>	<code>IFNULL(x, 'default')</code> or <code>COALESCE</code> (both work)	[Depth] MySQL supports <code>COALESCE</code> too; <code>IFNULL</code> is MySQL-only shorthand for the two-argument case
Q71-035	Auto-incrementing primary key	<code>SERIAL / GENERATED ALWAYS AS IDENTITY</code>	<code>AUTO_INCREMENT</code>	[Business] copy-pasting a <code>CREATE TABLE</code> between engines without translating this is one of the most common cross-dialect errors
Q71-036	Limiting + skipping rows	<code>LIMIT n OFFSET m</code>	<code>LIMIT m, n</code> (reversed order!) or <code>LIMIT n OFFSET m</code> (also supported)	[Edge cases] MySQL's <code>LIMIT m, n</code> puts the offset <i>first</i> , the opposite order from what the words suggest: a common source of an off-by-swap bug when porting a query
Q71-037	Current date/time	<code>CURRENT_DATE</code> , <code>NOW()</code>	<code>CURDATE()</code> , <code>NOW()</code>	[Trade-offs] <code>NOW()</code> works identically on both; date-only needs the engine-specific function
Q71-038	Case sensitivity of identifiers	Unquoted identifiers are folded to lowercase	Table names are case-sensitive on Linux (filesystem-dependent), case-insensitive on Windows/Mac by default	[Edge cases] a query that works locally on a Mac can fail on a Linux production MySQL server purely from table-name casing: a genuinely common deployment surprise
Q71-039	Boolean type	Native <code>BOOLEAN</code> , <code>true/false</code>	No true <code>BOOLEAN</code> type; <code>TINYINT(1)</code> conventionally used instead	[Depth] MySQL's "boolean" columns are really just an integer convention, which is why <code>0 / 1</code> and <code>TRUE / FALSE</code> are interchangeable there in a way they aren't in stricter engines
Q71-040	Window functions	Full support since PostgreSQL 8.4	Full support since MySQL 8.0 (not in 5.7 or earlier)	[Business] if a company mentions an older MySQL version in the JD or in the interview, ask directly whether window functions are available before assuming this chapter's window-function questions (Q71-019 through Q71-026, Q71-031, Q71-032) all apply as written

71.10 Query optimization and indexes

Q71-041 · What does `EXPLAIN` show, and what did it show on Riverstone's own orders table?

Remember it as: `EXPLAIN` tells you what the database actually plans to do, not what you assume it does.

Answer in one line: `EXPLAIN` shows the query planner's chosen execution strategy (a sequential scan, an index scan, a join algorithm) and its estimated cost, before running the query.

Verified, live, on Riverstone's real orders table (175 rows):

```
EXPLAIN SELECT * FROM orders WHERE customer_id = 1;
```

QUERY PLAN

```
-----
Seq Scan on orders (cost=0.00..4.19 rows=16 width=25)
  Filter: (customer_id = 1)
(2 rows)
```

And after adding an index on `customer_id`, the plan doesn't change (re-run separately after `CREATE INDEX`, not re-verified inline here since a schema change isn't re-runnable in this checker):

QUERY PLAN

```
-----
Seq Scan on orders (cost=0.00..4.19 rows=16 width=25)
  Filter: (customer_id = 1)
(2 rows)
```

Tier	What to say
Passes	"EXPLAIN shows you the query plan"
Strong	+ explains what a sequential scan means (reads every row) versus an index scan (jumps directly to matching rows)
Extra points	+ [Validate] the real, live surprise above: even <i>with</i> an index in place, PostgreSQL's planner still chose a sequential scan on this table, and that's not a bug: on a 175-row table, scanning everything is genuinely cheaper than the overhead of using an index, and the planner correctly knows it + [Depth] this is the honest answer to "why isn't my index being used": indexes pay off at real scale (tens of thousands of rows and up), not on small tables, and a planner that ignores a small table's index is doing its job correctly, not failing

Likely follow-ups: At what table size would you expect the planner to switch to using the index? What's the difference between a sequential scan and an index scan in terms of actual disk I/O? **Red flag:** assuming an index is always used just because it exists. **Learn it in:** Chapter 12 (and Part V for indexing at production scale).

Rapid-fire, 71.7

#	Question	One-line answer	Extra point
Q71-042	What column(s) usually benefit most from an index?	Columns frequently used in <code>WHERE</code> , <code>JOIN ON</code> , and <code>ORDER BY</code> clauses, especially on large tables	[Trade-offs] every index speeds up reads but slows down writes (<code>INSERT</code> / <code>UPDATE</code> / <code>DELETE</code> must maintain it too): indexing everything is not free
Q71-043	What's a composite index, and when does column order matter?	An index on multiple columns together; it's most useful for queries filtering on the <i>leading</i> column(s), in the order the index was built	[Edge cases] an index on <code>(customer_id, order_date)</code> speeds up <code>WHERE customer_id = X AND order_date > Y</code> , but doesn't help a query filtering on <code>order_date</code> alone
Q71-044	What's a covering index?	An index that includes every column a query needs, letting the database answer entirely from the index without touching the actual table rows	[Business] this is one of the highest-impact, lowest-effort performance fixes for a frequently-run report query

71.11 Live-coding walk-throughs

Q71-045 · Walk-through: revenue by month, with a fair fan-out warning

What they're really testing: whether you notice a join that silently multiplies rows before you trust the total it produces.

The setup, talked through live: "I need monthly revenue. Orders join to order_items for the line-level amounts. If this business also has invoices and payments tables, and I'm tempted to join through both to get to 'paid revenue,' I need to check first whether an order can have more than one payment, because that join would then double-count the order's revenue once per payment."

Verified, live, on the real problem: Riverstone's mini database has exactly this shape: order 5006 has 2 payment rows.

```
-- WRONG: joining through invoices and payments before aggregating line items
SELECT o.order_id, SUM(oi.quantity*oi.unit_price*(1-oi.discount_pct/100)) AS wrong_total
FROM orders o JOIN order_items oi ON o.order_id = oi.order_id
JOIN invoices i ON o.order_id = i.order_id JOIN payments p ON i.invoice_id = p.invoice_id
WHERE o.order_id = 5006 GROUP BY o.order_id;
-- -> ₹29,280.00

-- RIGHT: aggregate line items first, independent of payments entirely
SELECT o.order_id, SUM(oi.quantity*oi.unit_price*(1-oi.discount_pct/100)) AS correct_total
FROM orders o JOIN order_items oi ON o.order_id = oi.order_id
WHERE o.order_id = 5006 GROUP BY o.order_id;
-- -> ₹14,640.00
```

The order's revenue is genuinely ₹14,640. The fan-out join reports ₹29,280: exactly double, because that order has exactly 2 payments.

Extra-points moves demonstrated: [Edge cases] anticipated the fan-out before running anything, from knowing orders can have multiple payments. [Validate] proved it with a real, live, doubled number on this exact order, not a hypothetical. [Business] stated the fix plainly: compute revenue from order_items alone, and join to payments (if needed at all) only for payment-status questions, never in the same aggregation as the revenue sum.

Likely follow-ups: How would you check for this kind of fan-out risk *before* writing the query, on a schema you've never seen? How would you fix a report that's already been shipped with this bug? **Red flag:** trusting a multi-join total without asking whether any of the joined tables could have more than one matching row per order. **Learn it in:** Chapter 12 (join cardinality) and Chapter 13.

Q71-046 · Walk-through: "give me our top customers": the clarifying-question round

What they're really testing: whether you ask before you build, on a question that's genuinely ambiguous.

Talked through live: "'Top customers' could mean several different things, and each needs a different query: by total revenue all-time, by revenue this year only, by order count, or by average order value. I'll ask which, and while I wait I'll build the total-revenue-all-time version, since that's the most common default meaning."

```

SELECT c.customer_name,
       SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct/100)) AS total_revenue
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
WHERE o.status <> 'Cancelled'
GROUP BY c.customer_name
ORDER BY total_revenue DESC
LIMIT 5;

```

Extra-points moves demonstrated: [Clarify] named the real ambiguity instead of guessing silently. [Simple first] still produced a working, reasonable-default answer while waiting, rather than stalling entirely on the clarifying question. [Edge cases] excluded cancelled orders from revenue, a decision stated out loud rather than silently assumed.

Likely follow-ups: Now do it by average order value instead. Now for a specific date range. Now excluding one large outlier customer: how would you handle that request? **Red flag:** picking one interpretation silently and never mentioning the ambiguity existed. **Learn it in:** Chapter 12.

Q71-047B · Walk-through: does each customer's spend beat their own prior trend?

What they're really testing: whether you can write a self-comparison (this period vs. this same customer's own history) without a self-join, using a window frame clause most candidates have only seen the default version of.

Talked through live: "I need each customer-month's revenue, then each row's average of *that customer's own* prior months only, not including the current one. A plain `AVG() OVER (PARTITION BY customer)` would include the current row and leak the answer into its own comparison. I need an explicit frame: everything before the current row, nothing after."

Verified, live:

```

WITH monthly AS (
  SELECT o.customer_id, date_trunc('month', o.order_date)::date AS month,
         SUM(oi.quantity*oi.unit_price*(1-oi.discount_pct/100)) AS revenue
  FROM orders o JOIN order_items oi ON o.order_id = oi.order_id
  WHERE o.status <> 'Cancelled' GROUP BY 1, 2
),
with_avg AS (
  SELECT customer_id, month, revenue,
         AVG(revenue) OVER (PARTITION BY customer_id ORDER BY month
                           ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING) AS prior_avg
  FROM monthly
)
SELECT customer_id, month, revenue::numeric(10,2) AS revenue, prior_avg::numeric(10,2) AS prior_avg
FROM with_avg
WHERE prior_avg IS NOT NULL AND revenue > prior_avg
ORDER BY customer_id, month LIMIT 3;

```

customer_id	month	revenue	prior_avg
1	2025-03-01	64387.50	63656.25
1	2025-10-01	96825.00	34443.89
1	2025-11-01	82250.00	40682.00

Note the fix mid-draft: the first version of this query put the window function directly in `WHERE` (`WHERE revenue > (AVG(...) OVER (...))`), exactly the mistake Q71-026 warns about, caught by this chapter's own formal re-verification step, not by careful writing the first time. Wrapping the window function in a CTE (`with_avg`) and filtering the *outer* query fixes it, the same pattern Q71-019 and Q71-021 already use. Left the correction visible here rather than quietly rewriting it, since catching your own version of a mistake you just taught is itself worth showing.

Extra-points moves demonstrated: **[Edge cases]** explicitly caught the default-frame trap before writing any code, not after getting a wrong answer. **[Validate]** customer 1's October revenue (₹96,825) against a prior average of only ₹34,444, a real, large beat, sanity-checked as plausible rather than assumed correct. **[Business]** this exact pattern, "is this customer trending up against their own history," is a genuinely useful account-health signal, distinct from comparing customers to each other.

Likely follow-ups: What if you wanted a 3-month trailing average instead of all prior months? How would this change using `LAG` instead of a frame clause? **Red flag:** using the default window frame and not noticing the current row leaking into its own comparison average. **Learn it in:** Chapter 13.

Q71-047C · Walk-through: pivot monthly revenue by category into columns, live

What they're really testing: whether conditional aggregation (`CASE` inside `SUM`) comes to mind, since standard SQL has no native `PIVOT` keyword the way some spreadsheet tools and a few specific database platforms do.

Talked through live: "There's no universal `PIVOT` in standard SQL. The standard trick is conditional aggregation: one `SUM(CASE WHEN category = X THEN revenue ELSE 0 END)` per category I want as its own column."

Verified, live:

```
SELECT date_trunc('month', o.order_date)::date AS month,
       SUM(CASE WHEN p.category='Storage' THEN oi.quantity*oi.unit_price*(1-oi.discount_pct/100) ELSE
0 END)::numeric(10,2) AS storage,
       SUM(CASE WHEN p.category='Kitchen' THEN oi.quantity*oi.unit_price*(1-oi.discount_pct/100) ELSE
0 END)::numeric(10,2) AS kitchen,
       SUM(CASE WHEN p.category='Industrial' THEN oi.quantity*oi.unit_price*(1-oi.discount_pct/100) ELSE
0 END)::numeric(10,2) AS industrial
FROM orders o JOIN order_items oi ON o.order_id = oi.order_id JOIN products p ON oi.product_id =
p.product_id
WHERE o.status <> 'Cancelled' GROUP BY 1 ORDER BY 1 LIMIT 2;
```

month	storage	kitchen	industrial
2025-01-01	172240.00	30400.00	0.00
2025-02-01	150231.50	54152.50	49280.00

Extra-points moves demonstrated: [Edge cases] January's Industrial column correctly shows 0, not blank or an error, a real month with no Industrial sales, handled by the `ELSE 0` inside each `CASE`.

[Trade-offs] stated plainly that this approach only scales to a category list known in advance, hard-coded into the query; a genuinely dynamic set of categories needs either application-side pivoting or engine-specific dynamic SQL, not a single static query. **[Validate]** row totals were spot-checked against the plain `GROUP BY category` version from earlier in the chapter, and they reconcile.

Likely follow-ups: What if the list of categories isn't known ahead of time? How would you do this in a BI tool instead of raw SQL? **Red flag:** reaching for a non-standard `PIVOT` keyword without checking whether the specific engine actually supports it. **Learn it in:** Chapter 12 and 13 (and Chapter 70, §70.3, for the same pivot idea done in Power Query).

Q71-047D · Walk-through: profile a table you've never seen before, in one query

What they're really testing: whether you have a fast, repeatable first move for an unfamiliar table, rather than guessing or scrolling through raw rows.

Talked through live: "Before I write anything specific, I want a fast health check: row count, how many NULLs in the columns that matter, the distinct count of what should be a key, and the date range if there's a date column. One query, several aggregates at once."

Verified, live, on Riverstone's orders table:

```
SELECT
  COUNT(*) AS total_rows,
  COUNT(*) - COUNT(sales_rep_id) AS null_sales_rep,
  COUNT(DISTINCT customer_id) AS distinct_customers,
  MIN(order_date) AS earliest, MAX(order_date) AS latest,
  COUNT(*) FILTER (WHERE status = 'Cancelled') AS cancelled_count
FROM orders;
```

total_rows	null_sales_rep	distinct_customers	earliest	latest	cancelled_count
175	11	23	2025-01-02	2025-12-23	2

Extra-points moves demonstrated: [Business] this single query answers, in seconds, several questions that would otherwise take five separate ones: is the table roughly the size I expect, is there missing data I need to account for, does the date range match what I was told, are there statuses (like Cancelled) I need to filter in every later query. **[Depth]** `COUNT(*) - COUNT(col)` as a NULL count and `COUNT(*) FILTER (WHERE ...)` as a conditional count are both small, reusable idioms worth having memorized, not reconstructed each time. **[Validate]** the 11 NULL `sales_rep_id` rows matches exactly the number this chapter's own NULL-trap questions (Q71-002) already found independently, a small internal consistency check that the profiling query itself is trustworthy.

Likely follow-ups: How would you extend this to profile every column in a table automatically? What would you check differently for a table with no natural date column? **Red flag:** starting with `SELECT * LIMIT 10` and eyeballing it as the only exploration step, with no aggregate health check at all. **Learn it in:** Chapter 12.

Common mistakes and how to spot them

Mistake	Symptom	Fix
<code>NOT IN</code> with a subquery that can contain NULL	Query silently returns zero rows	Filter NULLs in the subquery, or use <code>NOT EXISTS</code> / <code>LEFT JOIN ... IS NULL</code>
Filtering <code>WHERE</code> on an aggregate	Query errors	Use <code>HAVING</code> for aggregate conditions
<code>LIMIT / OFFSET</code> for "Nth highest"	Silently wrong on a tie	<code>DENSE_RANK</code> instead
Joining through a one-to-many table before aggregating	Totals inflate (fan-out), exactly Q71-041's doubled revenue	Aggregate the base fact table first; join dimension/detail tables that don't fan out
Assuming an index is always used	Confusion when <code>EXPLAIN</code> shows a sequential scan anyway	Check table size; small tables genuinely don't benefit
Forgetting <code>ORDER BY</code> inside a window function	<code>LAG / LEAD</code> /running totals become meaningless or nondeterministic	Every window function needs an explicit row order to mean anything
Assuming PostgreSQL and MySQL syntax is interchangeable	A query that works on one engine errors or misbehaves on the other	Check the specific dialect differences (§71.9) before porting

In the real world: the interview that hinged on one word

Deepak, interviewing for an Analytics Engineer role, is given a live SQL round: "Write a query showing each customer's total spend." He writes a clean, correct query joining customers to orders to `order_items`, summing revenue, in under three minutes.

The interviewer says: "Good. Now, what if I told you a customer can have order line items marked as returned, and 'total spend' should exclude those?" Deepak hadn't seen a `returned` flag in the schema he was shown, so instead of guessing, he asks: "Is there a status or flag column on `order_items` I should filter on, or does 'returned' show up somewhere else, like a separate returns table?" The interviewer, satisfied, describes a `returned` boolean column Deepak hadn't been shown, and Deepak adds one line: `WHERE returned = FALSE` (or `AND NOT oi.returned`, depending on how it's stored), explaining the change out loud as he types it.

What made this a strong round wasn't the original query, it was correct and unremarkable. It was that Deepak's first reaction to new information was a clarifying question, not a guess, and once given the real answer, resumed working with essentially no wasted motion, since the underlying query structure didn't need to change, only one filter needed to be added. That's Chapter 69's Move 1, live, under real pressure, and it's the single most common thing separating a "pass" from a "strong pass" in this specific kind of round.

Tools

PostgreSQL and **MySQL**, both free, both open source. `psql` (PostgreSQL's command-line client) and the `mysql` client, or a GUI like DBeaver or TablePlus for either. `EXPLAIN` (both engines) and `EXPLAIN ANALYZE` (PostgreSQL; actually runs the query and reports real timings, not just estimates) for understanding query plans. Everything in this chapter was run against PostgreSQL 16.15 and MySQL 8.0.46, on Riverstone's `riverstone` and `riverstone_2025` practice databases.

The project

Goal: verified, real answers to the classic SQL problems, on your own data or Riverstone's.

1. Write and run the "customers with no orders" query (Q71-001), and reconcile the count against total customers minus customers-with-orders, the way this chapter did.
 2. Demonstrate the `NOT IN NULL` trap yourself: find a nullable foreign key in your data, and show the same query returning zero rows with `NOT IN` and the correct count with `NOT EXISTS`.
 3. Write one "top N per group" window-function query, and check whether any group returns fewer than N rows, honestly reporting it if so.
 4. Run `EXPLAIN` on one of your own queries before and after adding an index, and report what actually changed, not what you expected to change.
 5. Find one place in your own data (or Riverstone's) where a join could fan out unexpectedly, and prove the doubled/tripled total the way Q71-041 did.
-

Final-week revision list

Q71-001, Q71-002, Q71-006, Q71-009, Q71-014, Q71-015, Q71-016, Q71-019, Q71-020, Q71-021, Q71-022, Q71-028, Q71-029, Q71-033, Q71-036, Q71-040, Q71-041, Q71-045, Q71-046.

Key terms

`LEFT JOIN` / `INNER JOIN` · `NOT IN NULL` trap · `NOT EXISTS` · `WHERE` vs. `HAVING` · correlated subquery · scalar subquery · CTE (`WITH`) · recursive CTE · `EXISTS` vs. `IN` · `ROW_NUMBER` / `RANK` / `DENSE_RANK` · `PARTITION BY` · `LAG` / `LEAD` · running total · gaps-and-islands · fan-out (join cardinality) · `EXPLAIN` · sequential scan vs. index scan · composite index · covering index · `STRING_AGG` / `GROUP_CONCAT` · `COALESCE` / `IFNULL` · `SERIAL` / `AUTO_INCREMENT`

Where this leads

- **Chapter 69, The Extra-Points Method**, is the rubric and move set every answer above is written against.
- **Chapter 70, Excel/Sheets/VBA/BI Question Bank**, picks up the same Merge/Append and `QUERY` logic in spreadsheet form.
- **Chapter 77, Data Engineering & Data System Design Bank**, takes indexing and query optimization (§71.10) to genuine production scale.
- **Chapters 12 and 13** teach every technique this bank draws on, in full; this chapter tests it, it doesn't re-teach it from scratch.